

read json file

```
In [4]: import pandas as pd  
df=pd.read_json("titaanic.json")  
df.head(10)
```

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2835
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708



In [7]:

```
import pandas as pd
df=pd.read_excel("Supermarket-Sales-Sample-Data.xlsx")
```

```
df.tail(5)
```

Out[7]:

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7
72	NaN	1066	2024-02-02 00:00:00	Sarah Gonzalez	2024-02-06 00:00:00	79.99	2	1
73	NaN	1067	2024-02-03 00:00:00	Matthew Smith	2024-02-06 00:00:00	49.99	3	1
74	NaN	1068	2024-02-04 00:00:00	Emily Johnson	2024-02-05 00:00:00	129.99	1	1
75	NaN	1069	2024-02-04 00:00:00	Daniel Brown	2024-02-08 00:00:00	19.99	4	1
76	NaN	1070	2024-02-04 00:00:00	Jennifer Hernandez	2024-02-07 00:00:00	149.99	1	1



```
In [9]: import pandas as pd
df = pd.read_json("titaanic.json")
df.head()
#check missing values
#isnull()
#used to identify missing values
print(df.isnull())
#count missing values for all field
df.isnull().sum()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	\
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	
..	...	...	...	...	...	...	...	...	...	
886	False	False	False	False	False	False	False	False	False	
887	False	False	False	False	False	False	False	False	False	
888	False	False	False	False	False	True	False	False	False	
889	False	False	False	False	False	False	False	False	False	
890	False	False	False	False	False	False	False	False	False	

	Fare	Cabin	Embarked
0	False	True	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	True	False
..	...	...	...
886	False	True	False
887	False	False	False
888	False	True	False
889	False	False	False
890	False	True	False

[891 rows x 12 columns]

```
Out[9]: PassengerId      0
        Survived        0
        Pclass          0
        Name            0
        Sex             0
        Age             177
        SibSp           0
        Parch           0
        Ticket          0
        Fare            0
        Cabin           687
        Embarked        2
        dtype: int64
```

```
In [13]: #Find Rows Having Missing Age
print(df[df['Age'].isnull()])

print(df.columns)

#fillna
#replace missing values.
#Fill Age With Mean
mean_age = df['Age'].mean()
df['Age'] = df['Age'].fillna(mean_age)
#count missing values for all field
df.isnull().sum()

#Fill embarked with most frequent values
mode_embarked = df['Embarked'].mode()[0]
```

```

df['Embarked'] = df['Embarked'].fillna(mode_embarked)

#dropna
#remove missing data
#remove rows
df.dropna()

#remove column
df.dropna(axis=1)
print(df.isnull().sum())

```

Empty DataFrame

Columns: [PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked]

Index: []

Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
dtype='object')

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	0

dtype: int64

In [17]: *#you need specific Row and Column*

```

print(df.iloc[0:5])
print(df.iloc[0:5,1:3])

#change my index to customer id
df = pd.read_csv('customers-100.csv',index_col='Customer Id')
print(df)

#Accessing specific column for particular index
print(df.loc['6F94879bDAfE5a6',['First Name','Company']])

#range from this to that
#range from this to that
print(df.loc['DD37Cf93aecA6Dc':'5Cef8BFA16c5e3c',['First Name','Country']])

#groupby
#Average Fare for Class
print(df.groupby('Pclass')['Fare'].mean())
#Count Survivors By Gender
df.groupby('sex')['Survived'].sum()

#multiple Aggregations
#agg()

```

```
#Interpolate()
#Estimate missing values.
sample = pd.DataFrame({
    'Age': [20, 25, None, 35, 40]
})
sample['Age'] = sample['Age'].interpolate()
print(sample)
```

Customer Id	Index	First Name	Last Name	Company \
DD37Cf93aecA6Dc	1	Sheryl	Baxter	Rasmussen Group
1Ef7b82A4CAAD10	2	Preston	Lozano	Vega-Gentry
6F94879bDAfE5a6	3	Roy	Berry	Murillo-Perry
5Cef8BFA16c5e3c	4	Linda	Olsen	Dominguez, Mcmillan and Donovan
053d585Ab6b3159	5	Joanna	Bender	Martin, Lang and Andrade

Customer Id	City	Country \
DD37Cf93aecA6Dc	East Leonard	Chile
1Ef7b82A4CAAD10	East Jimmychester	Djibouti
6F94879bDAfE5a6	Isabelborough	Antigua and Barbuda
5Cef8BFA16c5e3c	Bensonview	Dominican Republic
053d585Ab6b3159	West Priscilla	Slovakia (Slovak Republic)

Customer Id	Phone 1	Phone 2 \
DD37Cf93aecA6Dc	229.077.5154	397.884.0519x718
1Ef7b82A4CAAD10	5153435776	686-620-1820x944
6F94879bDAfE5a6	+1-539-402-0259	(496)978-3969x58947
5Cef8BFA16c5e3c	001-808-617-6467x12895	+1-813-324-8756
053d585Ab6b3159	001-234-203-0635x76146	001-199-446-3860x3486

Customer Id	Email	Subscription Date \
DD37Cf93aecA6Dc	zunigavanessa@smith.info	2020-08-24
1Ef7b82A4CAAD10	vmata@colon.com	2021-04-23
6F94879bDAfE5a6	beckycarr@hogan.com	2020-03-25
5Cef8BFA16c5e3c	stanleyblackwell@benison.org	2020-06-02
053d585Ab6b3159	colinalvarado@miles.net	2021-04-17

Customer Id	Website
DD37Cf93aecA6Dc	http://www.stephenson.com/
1Ef7b82A4CAAD10	http://www.hobbs.com/
6F94879bDAfE5a6	http://www.lawrence.com/
5Cef8BFA16c5e3c	http://www.good-lyons.com/
053d585Ab6b3159	https://goodwin-ingram.com/

Customer Id	First Name	Last Name
DD37Cf93aecA6Dc	Sheryl	Baxter
1Ef7b82A4CAAD10	Preston	Lozano
6F94879bDAfE5a6	Roy	Berry
5Cef8BFA16c5e3c	Linda	Olsen
053d585Ab6b3159	Joanna	Bender

Customer Id	Index	First Name	Last Name	Company \
DD37Cf93aecA6Dc	1	Sheryl	Baxter	Rasmussen Group
1Ef7b82A4CAAD10	2	Preston	Lozano	Vega-Gentry
6F94879bDAfE5a6	3	Roy	Berry	Murillo-Perry
5Cef8BFA16c5e3c	4	Linda	Olsen	Dominguez, Mcmillan and Donovan
053d585Ab6b3159	5	Joanna	Bender	Martin, Lang and Andrade
...	...	...	...	...
cb8E23e48d22Eae	96	Karl	Greer	Carey LLC
CeD220bdAaCfaDf	97	Lynn	Atkinson	Ware, Burns and Oneal

28CDbC0dFe4b1Db	98	Fred	Guerra	Schmitt-Jones
c23d1D9EE8DEB0A	99	Yvonne	Farmer	Fitzgerald-Harrell
2354a0E336A91A1	100	Clarence	Haynes	Le, Nash and Cross

Customer Id	City	Country \
DD37Cf93aecA6Dc	East Leonard	Chile
1Ef7b82A4CAAD10	East Jimmychester	Djibouti
6F94879bDAfE5a6	Isabelborough	Antigua and Barbuda
5Cef8BFA16c5e3c	Bensonview	Dominican Republic
053d585Ab6b3159	West Priscilla	Slovakia (Slovak Republic)
...	...	...
cb8E23e48d22Eae	East Richard	Guyana
CeD220bdAaCfaDf	New Bradview	Sri Lanka
28CDbC0dFe4b1Db	Ortegaland	Solomon Islands
c23d1D9EE8DEB0A	Lake Elijahview	Aruba
2354a0E336A91A1	Judymouth	Honduras

Customer Id	Phone 1	Phone 2 \
DD37Cf93aecA6Dc	229.077.5154	397.884.0519x718
1Ef7b82A4CAAD10	5153435776	686-620-1820x944
6F94879bDAfE5a6	+1-539-402-0259	(496)978-3969x58947
5Cef8BFA16c5e3c	001-808-617-6467x12895	+1-813-324-8756
053d585Ab6b3159	001-234-203-0635x76146	001-199-446-3860x3486
...	...	...
cb8E23e48d22Eae	(188)169-1674x58692	001-841-293-3519x614
CeD220bdAaCfaDf	+1-846-706-2218	605.413.3198
28CDbC0dFe4b1Db	+1-753-067-8419x7170	+1-632-666-7507x92121
c23d1D9EE8DEB0A	(530)311-9786	001-869-452-0943x12424
2354a0E336A91A1	(753)813-6941	783.639.1472

Customer Id	Email Subscription Date \
DD37Cf93aecA6Dc	zunigavanessa@smith.info 2020-08-24
1Ef7b82A4CAAD10	vmata@colon.com 2021-04-23
6F94879bDAfE5a6	beckycarr@hogan.com 2020-03-25
5Cef8BFA16c5e3c	stanleyblackwell@benson.org 2020-06-02
053d585Ab6b3159	colinalvarado@miles.net 2021-04-17
...	...
cb8E23e48d22Eae	hhart@jensen.com 2022-01-30
CeD220bdAaCfaDf	vkemp@ferrell.com 2021-07-10
28CDbC0dFe4b1Db	swagner@kane.org 2021-09-18
c23d1D9EE8DEB0A	mccarthystephen@horn-green.biz 2021-08-11
2354a0E336A91A1	colleen91@faulkner.biz 2020-03-11

Customer Id	Website
DD37Cf93aecA6Dc	http://www.stephenson.com/
1Ef7b82A4CAAD10	http://www.hobbs.com/
6F94879bDAfE5a6	http://www.lawrence.com/
5Cef8BFA16c5e3c	http://www.good-lyons.com/
053d585Ab6b3159	https://goodwin-ingram.com/
...	...
cb8E23e48d22Eae	http://hayes-perez.com/
CeD220bdAaCfaDf	https://novak-allison.com/

28CDbC0dFe4b1Db <https://www.ross.com/>
c23d1D9EE8DEB0A <http://watkins.info/>
2354a0E336A91A1 <http://www.hatfield-saunders.net/>

[100 rows x 11 columns]

First Name Roy

Company Murillo-Perry

Name: 6F94879bDAfE5a6, dtype: object

First Name

Country

Customer Id

DD37Cf93aecA6Dc Sheryl Chile

1Ef7b82A4CAAD10 Preston Djibouti

6F94879bDAfE5a6 Roy Antigua and Barbuda

5Cef8BFA16c5e3c Linda Dominican Republic

```

-----
KeyError                                Traceback (most recent call last)
Cell In[17], line 19
    14 print(df.loc['DD37Cf93aecA6Dc':'5Cef8BFA16c5e3c',['First Name','Country']])
    17 #groupby
    18 #Average Fare for Class
--> 19 print(df.groupby( )['Fare'].mean())
    20 #Count Survivors By Gender
    21 df.groupby('sex')['Survived'].sum()

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:9210, in DataFrame.groupby(self, by, axis, level, as_index, sort, group_keys, observed, dropna)
    9207 if level is None and by is None:
    9208     raise TypeError("You have to supply one of 'by' and 'level'")
-> 9210 return DataFrameGroupBy(
    9211     obj=self,
    9212     keys=by,
    9213     axis=axis,
    9214     level=level,
    9215     as_index=as_index,
    9216     sort=sort,
    9217     group_keys=group_keys,
    9218     observed=observed,
    9219     dropna=dropna,
    9220 )

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\groupby.py:1331, in GroupBy._init__(self, obj, keys, axis, level, grouper, exclusions, selection, as_index, sort, group_keys, observed, dropna)
    1328 self.dropna = dropna
    1330 if grouper is None:
-> 1331     grouper, exclusions, obj = get_grouper(
    1332         obj,
    1333         keys,
    1334         axis=axis,
    1335         level=level,
    1336         sort=sort,
    1337         observed=False if observed is lib.no_default else observed,
    1338         dropna=self.dropna,
    1339     )
    1341 if observed is lib.no_default:
    1342     if any(ping._passed_categorical for ping in grouper.groupings):

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\grouper.py:1043, in get_grouper(obj, key, axis, level, sort, observed, validate, dropna)
    1041     in_axis, level, gpr = False, gpr, None
    1042     else:
-> 1043     raise KeyError(gpr)
    1044 elif isinstance(gpr, Grouper) and gpr.key is not None:
    1045     # Add key to exclusions
    1046     exclusions.add(gpr.key)

KeyError: 'Pclass'

```

In []: